COMPUTER GRAPHICS | GAME ENGINE ENGINEERING

Adaptive Level of Detail for Real-Time Rendering in Godot

A feedback-controlled, distance-based LOD for vegetation, compared against no LOD (full detail)

Shreeniwas Salian

Problem Dense vegetation scenes cost frame time; fixed LOD distances cannot react when the GPU is over budget, so detail is either wasted or cut blindly.



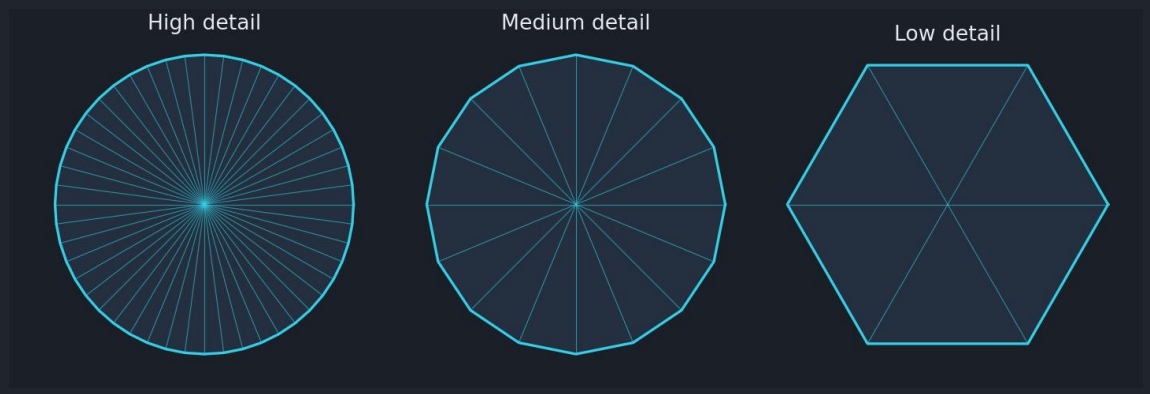
Real frame captured from the running project (Godot 4.7.2, Terrain3D 1.0.2, 1920x1080, Adaptive mode).

41,698
vegetation instances

528 m
deterministic camera path

2 modes
none / adaptive

Why LOD, and where fixed thresholds fall short



Schematic only: polygon count falls as distance grows (not the project meshes).

LOD0	LOD1	LOD2
0 to 60 m	60 to 150 m	beyond 150 m

Starting tree thresholds from the code (60 m / 150 m, 10 m hysteresis); the bias scales them.

General LOD concepts

- Cost scales with triangles, draw calls and overdraw
- Distant objects need fewer triangles for the same look
- Fixed distances ignore the actual frame budget

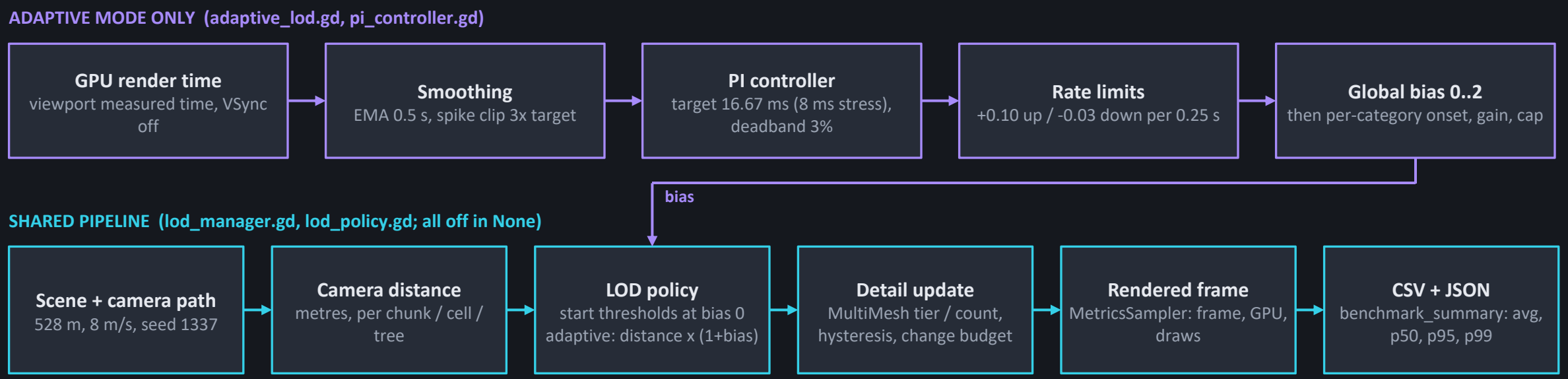
Implemented in this project

- Distance LOD for 4 categories: trees (3 mesh levels), grass (3 tiers + cull), bushes and debris (density thinning)
- Adaptive: one PI-controlled bias scales those thresholds from measured GPU render time
- Hysteresis, rate limits and a per-update change budget limit popping
- Same scene, seed and camera path for None and Adaptive

Not implemented

- Per-instance or screen-size importance
- Cross-fade or dither between mesh levels (trees still pop)
- Mesh LOD for bushes (density only)

Architecture: one controller bias scales fixed metre thresholds



Thresholds in metres

- Trees: effective = max(floor, base / (1 + bias)), never above base
- Hysteresis added in metres (10 m)
- Bias 0 gives the starting thresholds, so one code path serves every bias

Bounded coarsening

- Tree floors: LOD1 never before 60 m, LOD2 never before 100 m
- Per-category caps: grass 1.0, trees 1.0, bushes 0.5, debris 0.5
- bias_min 0: never finer than baseline. bias_max 2.0

baseline_lod.gd: _rebuild_thresholds()

```
for i in 2:
  var effective: float = base[i] /
_dist_scale
  if _dist_scale > 1.0:  # coarser
    effective = minf(base[i],
      maxf(effective, floors[i]))
  _thresholds[i] = effective
```

NOVELTY

Technical contributions, and what the evidence supports

Contribution	None (no LOD) does	Adaptive changes	Why it matters	Evidence
1 Adaptive thresholds	Draws everything at full detail	One PI-controlled bias scales all thresholds from GPU time	Reacts to load instead of ignoring it	Live recording: bias rises from 0 to its 2.0 maximum while the panel shows the tree threshold move. Bias saturated in 84.3% of 1440p frames
2 Real-world units	No distance logic	Floors, caps and hysteresis in metres; coarser = distance x (1+bias)	Effect is predictable and tunable in scene units	Panel shows thresholds in m. Self-tests: 124 + 90 + 51 + 17 checks pass (grass determinism stage not run)
3 Constrained coarsening	No notion of a limit	Tree floors 60 / 100 m, category caps, bias range 0..2	Keeps nearby detail intact	At high bias the panel shows 60 / 100 m: LOD2 stops at the floor. Not literally a 2-level limit
4 Unified benchmark	n/a	Same scene, seed, path (sha256 logged), interleaved runs, CSV + JSON	Fair, repeatable comparison	30 runs, 5 per mode and resolution. Earlier sessions showed 2 to 3x GPU-state drift
5 Quality / speed trade-off	Full detail	Measures primitives, draws, GPU ms and visual difference	Shows what a saving costs	vs None: 77.9% (1080p) and 72.9% (1440p) lower frame time, 90% fewer primitives. No perceptual metric

Scope of the claim: these are this project's technical contributions. A limited literature check (slide 8) found no directly comparable controller-plus-vegetation result, but it was not a systematic review, so no global novelty is claimed.

METHODOLOGY

Experimental setup and fairness of the comparison

Condition	None (no LOD)	Adaptive
Build	Godot 4.7.2-stable DEBUG + EDITOR build (release export blocked: no export_presets.cfg and no export templates installed)	same binary
Hardware	NVIDIA Quadro P3200, driver 582.16; Intel i7-8850H, 64 GB; Windows, Balanced power plan	same machine
Renderer	Forward+ / D3D12, no MSAA / TAA, VSync off, FPS uncapped	same
Resolution	Normal 1920x1080 | Stress 2560x1440	same
Scene	Terrain3D 1.0.2, 41,698 instances + 403k grass clumps, seed 1337	same
Camera	528 m path (sha256 54ab438e9ad0), 8 m/s, 10 s warm-up, 60 s measured	same; controller reset to bias 0 at start
LOD	All LOD policies off: baked full-detail vegetation	Distance thresholds x PI bias; target 16.67 ms (normal), 8 ms (stress)
Runs	5 per mode and resolution, order rotated each repetition	5 per resolution
Metrics	Per-frame frame time, GPU ms, draws, primitives, switches, policy CPU ms	+ bias and controller terms

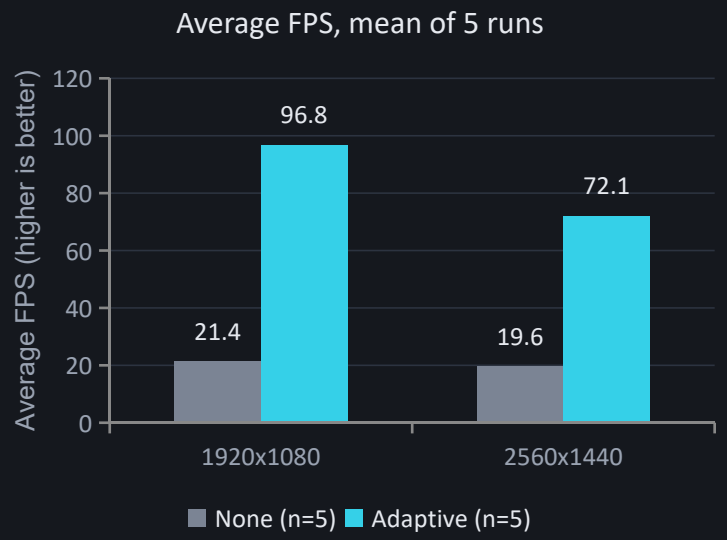
Fairness risks

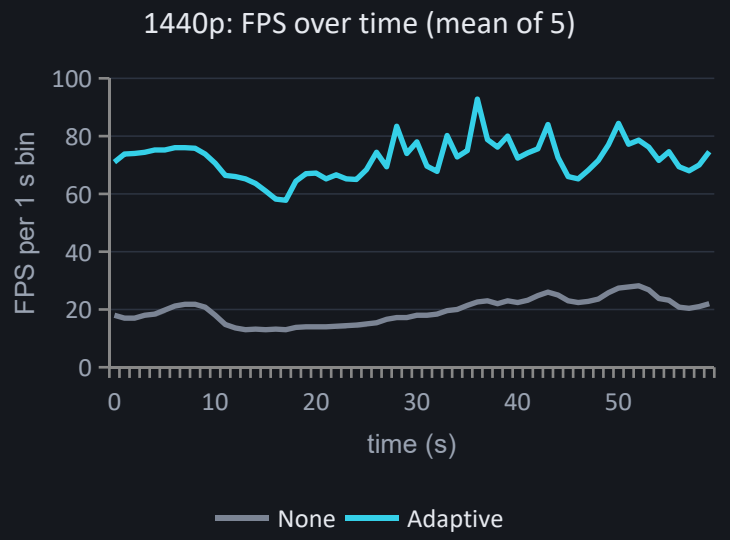
- GPU state drifted 2 to 3x between earlier sessions; this series stayed stable (sd of avg FPS 0.1 to 0.3)
- Debug + editor build inflates CPU timings
- Targets differ by profile, by design

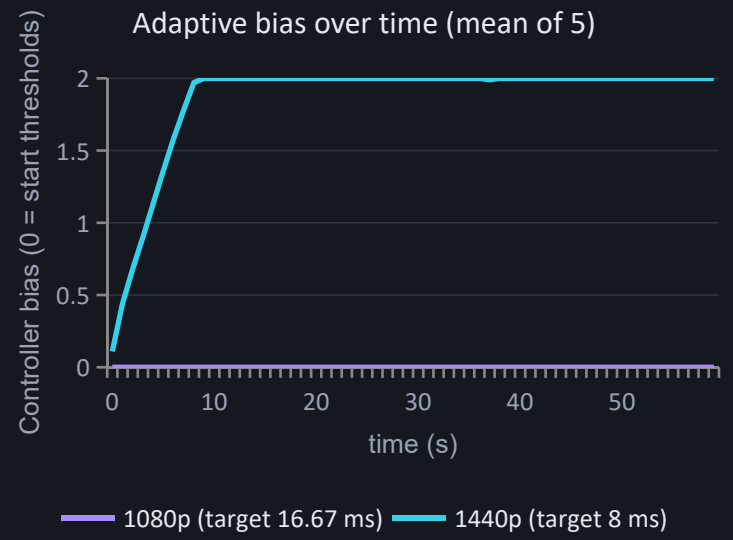
Not available

- Release / exported build results
- Control of GPU power state
- Grass determinism self-test stage (skipped by default)
- Recording ran with FFmpeg capture on: demo FPS is not benchmark data

Results: 4.5x the FPS at 1080p and 3.7x at 1440p versus no LOD







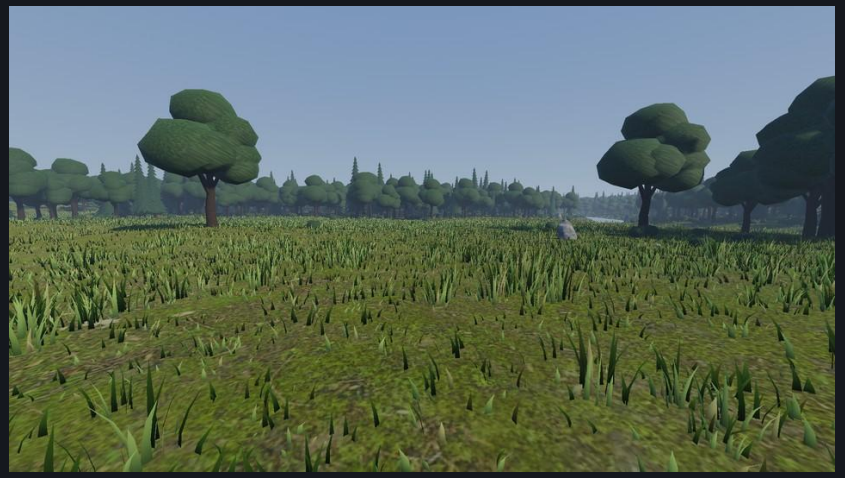
Resolution	Mode	Avg FPS ± sd	1% low FPS	Frame ms	GPU ms	FPS change	Frame-time change
1920x1080	None	21.4 ± 0.1	13.9	46.72	46.3	-	-
1920x1080	Adaptive	96.8 ± 0.3	77.6	10.33	9.9	**+352.3%**	**-77.9%**
2560x1440	None	19.6 ± 0.0	13.0	51.15	50.6	-	-
2560x1440	Adaptive	72.1 ± 0.1	57.3	13.87	13.3	**+268.9%**	**-72.9%**

Change = (Adaptive - None) / None x 100; 1% low = 1000 / p99 frame time. Debug/editor build. How much is the controller? Measured separately against the fixed-threshold LOD (not shown here): +0.2% avg FPS at 1080p, +2.4% at 1440p, so nearly all of the gain above comes from distance LOD itself.

Reading the result

- GPU time 46.3 to 9.9 ms (1080p) and 50.6 to 13.3 ms (1440p); primitives 11.2 M to 1.08 M, draw calls 3120 to 591
- 1080p: GPU is under the 16.67 ms target, bias stays 0. 1440p: bias saturates in 84% of frames and the 8 ms target is not reached
- 0 frames under 10 FPS in the 20 None/Adaptive runs; lowest per-run FPS 12.7

Visual comparison and live demonstration



None (no LOD)

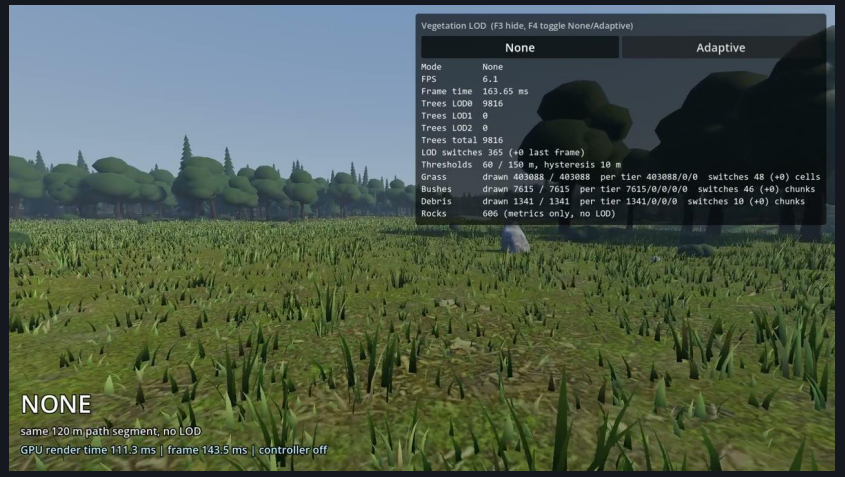


Adaptive (live controller, bias 1.4)

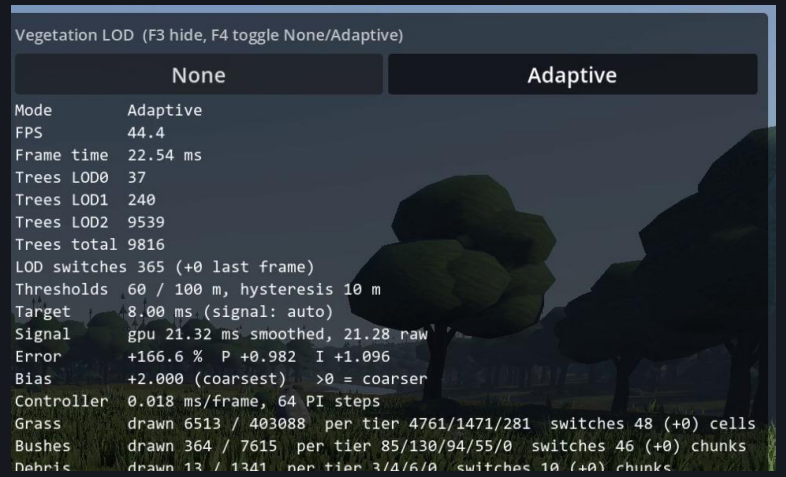




2x crop, same region: None (top), Adaptive (bottom)



Screen recording (70 s): None and Adaptive, F4 toggles



Real panel (F3): only None and Adaptive buttons

Stills: earlier session (GPU about 9.5 ms, bias 1.4). Video and panel: new recording after the Baseline button was removed; the GPU was slower (about 21 ms) so the bias hit its maximum of 2.0. Capture was running, so recording FPS is not benchmark data. Grass wind differs between frames; tree swaps still pop (no cross-fade).

How our results compare with recent research (2020 and later)

Work (year, venue)	Technique	Hardware and scene	Reported result	Relation to this project
Adaptive LOD / progressive-mesh control for stable real-time VR rendering (2026, The Visual Computer, DOI 10.1007/s00371-026-04784-5)	Frame-budget-driven adaptive LOD via progressive meshes	VR interior-design scenes, 11.11 ms budget (90 Hz)	High load: p95 GPU frame time 18.11 to 10.86 ms (-40%); frames over budget 84.51% to 7.01%, vs full-resolution rendering	Closest in idea (budget-driven LOD vs full detail). Ours: p99 frame time 72.0 to 12.9 ms (-82.1%) at 1080p, 77.0 to 17.4 ms (-77.4%) at 1440p
Real-Time GPU Tree Generation (2025, High-Performance Graphics, DOI 10.2312/hpg.20251168; AMD)	Procedural trees via DX12 work graphs and mesh nodes, continuous per-frame LOD	Radeon RX 7900 XTX, 1,200 trees of 20 types plus grass and effects	Median frame 7.74 ms for the whole scene (trees 3.13 ms to G-buffer); 51 KiB of data instead of 34.8 GiB of meshes	Different goal (memory and generation); shows trees at LOD fit in single-digit ms on far newer hardware. Ours: 10.3 ms (1080p) on a Quadro P3200 laptop GPU
Real-time Path Tracing of Massive Dynamic Foliage (ACM, DOI 10.1145/3820021)	Clustered LOD generation plus opacity micromaps for ray tracing	RTX 4090, animated foliage, 1440p upscaled to 2160p	About 60 FPS (no baseline comparison extracted)	Different pipeline (path tracing). Confirms LOD clustering for foliage remains an active research area; ours is raster, distance-based
Automatic LOD for Improved Rendering by CPU Usage (2024, RSU Int. Research Conf.)	Adaptive LOD updates, spatial hashing, CPU/GPU analysis for consistent FPS	Two test areas (Area 1, Area 2); hardware not stated	CPU time 15.1 to 14.2 ms (-5.96%) and 10.8 to 10.3 ms (-4.63%); triangles -15.79% and -25%; FPS 66.1 to 90 (+36.2%) and 121 to 146 (+20.7%)	Closest in aim. Like-for-like with our Baseline to Adaptive step (+0.2% / +2.4% FPS, primitives -0.2% / -7.9%): theirs is larger, CPU-driven; ours GPU-driven, GPU-bound scene
This project (2026)	PI-controlled bias on distance thresholds, 4 vegetation categories	Quadro P3200, Godot 4.7.2 debug build, 41,698 instances + 403k grass clumps	21.4 to 96.8 FPS (1080p), 19.6 to 72.1 FPS (1440p); GPU 46.3 to 9.9 ms and 50.6 to 13.3 ms; n=5 per cell	Gain vs None is large but mostly from distance LOD; controller adds +0.2% / +2.4%

Caveat Different hardware, scenes and baselines, so the numbers are context, not a like-for-like ranking. This was a limited web search, not a systematic review; papers that report comparable FPS or ms for LOD controllers on vegetation were scarce. Figures are as reported in the search results; verify against the papers before citing.

SUMMARY

Conclusion, limitations and next steps

Implemented

- Distance LOD for 4 vegetation categories, scaled by one PI-controlled bias
- PI controller with rate limits, deadband, anti-windup
- Metre-based floors keep near-field detail
- Benchmark harness, debug panel (None / Adaptive), 4 self-test suites with a log-validating runner

What the data shows

- 1080p: 21.4 to 96.8 FPS (4.5x), frame time 77.9% lower
- 1440p: 19.6 to 72.1 FPS (3.7x), frame time 72.9% lower
- Primitives about 90% lower; GPU time 78.7% / 73.7% lower
- The controller itself adds only +0.2% / +2.4% over fixed distances

Limitations

- Debug/editor build only; no release build possible
- GPU drifted 2 to 3x between sessions (the new demo recording ran in the slow state)
- No perceptual quality metric; tree pops remain
- No per-instance importance; grass determinism stage not run

Future work

Tune floors and caps, add cross-fade to remove pops, benchmark a release build with a GPU profiler, add a CPU-bound signal, extend runs across GPU power states, add SSIM or FLIP, and a systematic literature comparison.

Takeaway Compared with drawing everything, adaptive LOD runs 4.5x (1080p) and 3.7x (1440p) faster in this scene. Most of that is distance LOD; the feedback controller adds a bounded safety margin when the GPU is over its target.